

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Application-Based Questions

Weightage: 40%

QUESTIONS:

Total Marks: 30

1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
  <input type="text" name="name">
  <input type="email" name="email">
  <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly. **Questions:**

1. Identify appropriate **HTML5 semantic elements** missing in the structure.
2. Modify the structure to make it **standards-compliant**.
3. Interpret the **HTTP request generated** during form submission.
4. Analyze the **HTTP response cycle** from server to browser.
5. Suggest improvements for **usability and accessibility**.

2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
  <head>
    <title>Company</title>
  </head>
  <body>
    <div>
      <h1>Welcome
      <p>About us
    </div>
  </body> </html>
```

Questions:

1. Identify **improper nesting and missing closing tags**.
2. Analyze how different browsers may interpret this structure.
3. Identify **missing semantic elements** (e.g., header, section).
4. Provide a **corrected W3C-compliant HTML structure**.
5. Suggest techniques to ensure **cross-browser compatibility**.

3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

Questions:

1. Identify issues in the **form configuration**.
2. Analyze why the **HTTP request is not triggered**.
3. Interpret the role of **GET method in this scenario**.
4. Propose a **corrected implementation**.
5. Suggest improvements for **secure data transmission**.

Code of Conduct:

I Ch. Sri Lakshmi/192311342 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Ch. Sri Lakshmi

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

ANSWERS

1: Online Symposium Portal

1. Identify appropriate HTML5 semantic elements missing in the structure.

The following HTML5 semantic elements are missing:

- `<html lang="en">`
- `<head>` with `<meta charset="UTF-8">`
- `<meta name="viewport">`
- `<header>`
- `<main>`
- `<section>`

- <footer>
- <label> for input fields

These elements improve accessibility, SEO, readability, and standards compliance.

2. Modify the structure to make it standards-compliant.

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Symposium Registration</title>
</head>
<body>
<header>
<h1>Online Symposium Registration</h1>
</header>
<main>
<section>
<form action="/register" method="POST">
<label for="name">Name</label>
<input type="text" id="name" name="name" required>
<label for="email">Email</label>
<input type="email" id="email" name="email" required>
<button type="submit">Submit</button>
</form>
</section>
</main>

<footer>
<p>&copy; 2026 SIMATS</p>
</footer>
</body>
</html>
```

3. Interpret the HTTP request generated during form submission.

Since the form uses **POST**, the browser sends an HTTP POST request.

Example:

POST /register HTTP/1.1

Host: www.college.edu

Content-Type: application/x-www-form-urlencoded

[name=Sri&email=sri@gmail.com](#)

Explanation:

- **POST** sends data in the request body.
- The server receives the student's registration details.
- Sensitive data is not shown in the URL.

4. Analyze the HTTP response cycle from server to browser.

1. User clicks **Submit**.
2. Browser sends an HTTP POST request.
3. Server validates the data.
4. Server stores the registration details.
5. Server returns an HTTP response.
6. Browser displays a confirmation page like "**Registration Successful.**"

5. Suggest improvements for usability and accessibility.

- Use descriptive <label> tags.
- Add required validation.
- Display meaningful error messages.
- Use placeholder text.
- Ensure keyboard accessibility.
- Add ARIA attributes where needed.
- Use responsive design.
- Enable HTTPS for secure communication.

2: Cross-Browser Compatibility Issue

1. Identify improper nesting and missing closing tags.

Problems:

- <h1> is not closed.
- <p> is not closed.
- Missing <!DOCTYPE html>.
- Missing <meta charset>.
- Missing <html lang="en">.

Incorrect:

```
<h1>Welcome
```

```
<p>About us
```

Correct:

```
<h1>Welcome</h1>
```

```
<p>About us</p>
```

2. Analyze how different browsers may interpret this structure.

Different browsers automatically correct invalid HTML differently.

Possible effects:

- Broken page layout.
- Incorrect spacing.
- Text displayed in unexpected positions.
- CSS rendering differences.
- JavaScript may behave inconsistently.

Hence, invalid HTML causes inconsistent browser behavior.

3. Identify missing semantic elements.

Missing semantic tags include:

- `<header>`
- `<main>`
- `<section>`
- `<footer>`

These improve document structure and accessibility.

4. Provide a corrected W3C-compliant HTML structure.

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Company</title>
</head>
<body>
<header>
<h1>Welcome</h1>
</header>
<main>
<section>
<p>About us</p>
</section>
</main>
<footer>
<p>Copyright 2026</p>
</footer>
</body>
</html>
```

5. Suggest techniques to ensure cross-browser compatibility.

- Validate HTML using W3C Validator.

- Use HTML5 semantic elements.
- Write standard CSS.
- Test on Chrome, Firefox, Edge, and Safari.
- Use CSS reset or Normalize.css.
- Use feature detection instead of browser detection.
- Keep browsers updated.
- Avoid deprecated HTML tags.

3: Form Submission Failure

1. Identify issues in the form configuration.

Problems:

- Button type is
type="button"
instead of
type="submit"
- Password is sent using **GET**, which is insecure.
- No input validation.

2. Analyze why the HTTP request is not triggered.

The button is:

```
<button type="button">
```

A normal button does not submit the form.

Only

```
<button type="submit">
```

or JavaScript

```
form.submit();
```

can trigger form submission.

Therefore, clicking the button sends no HTTP request.

3. Interpret the role of GET method in this scenario.

GET:

- Appends data to the URL.
- Used mainly for retrieving data.
- Not suitable for passwords.
- Data can be cached and bookmarked.
- Less secure.

Example:

`/submit?username=John&password=12345`

This exposes sensitive information.

4. Propose a corrected implementation.

```
<!DOCTYPE html>

<html>

<body>

<form action="/submit" method="POST">

<label>Username</label>

<input type="text" name="username" required>

<label>Password</label>

<input type="password" name="password" required>

<button type="submit">Login</button>

</form>

</body>

</html>
```

5. Suggest improvements for secure data transmission.

- Use **POST** instead of GET.
- Enable **HTTPS**.

- Validate input on both client and server.
 - Use strong password policies.
 - Encrypt passwords before storing (hashing).
 - Protect against SQL Injection.
 - Implement CSRF protection.
 - Manage user sessions securely.
- 